

VetAssist AI

Copiloto Clínico e Inteligencia Artificial para Veterinaria

Fecha de Generación: 06/06/2026 | Entorno: Producción

1. DESCRIPCIÓN DEL DOMINIO Y CASOS DE USO

VetAssist AI es un sistema inteligente diseñado para asistir a profesionales de la medicina veterinaria (en particular canina y felina) en la toma de decisiones clínicas diarias, consulta rápida de manuales, esquemas de vacunación y dosificaciones de fármacos.

Casos de Uso Principales:

- Soporte Clínico en Tiempo Real: Consultas diagnósticas complejas resueltas en segundos basándose en manuales.
- Calculadora de Dosis Determinista: Cálculo exacto de mililitros o miligramos según el peso corporal.
- Tablas de Inmunización: Consulta interactiva y estructurada de calendarios preventivos por edad.
- Reportes Clínicos en PDF: Exportación formal de la consulta y sus fuentes citadas.

2. ARQUITECTURA GENERAL DEL SISTEMA

El sistema implementa una arquitectura modular de cuatro capas:

1. Capa de Presentación: Desarrollada en Streamlit (src/app.py) para proporcionar una GUI fluida y moderna.
2. Capa de Orquestación: Gestionada por LangGraph mediante un grafo de estados (StateGraph) que coordina los agentes.
3. Capa RAG (Generación Aumentada por Recuperación): Une la segmentación de texto, embeddings locales y re-ranking.
4. Capa de Persistencia: Una base de datos vectorial local basada en ChromaDB.

Los agentes implementados son:

- Retriever Agent: Su objetivo es optimizar y reformular la query para buscar el mejor contexto bibliográfico.
- Writer Agent: Su objetivo es redactar la respuesta clínica, verificar la suficiencia de la información y citar fuentes.

3. PIPELINE RAG Y BASE VECTORIAL LOCAL

Para garantizar precisión clínica y mitigar alucinaciones de la IA, implementamos un pipeline RAG local:

- Chunking Semántico por Secciones: Diseñamos un fragmentador adaptativo (SectionChunker) que corta los documentos respetando títulos y subtítulos (tomas lógicas) con un límite de 500 palabras y solape de 50 palabras.
- Generación de Embeddings: Usamos el modelo local 'paraphrase-multilingual-MiniLM-L12-v2' que mapea los textos a coordenadas de 384 dimensiones. Al ser local, es 100% privado y gratuito.
- Motor de Búsqueda Vectorial: ChromaDB actúa como almacenamiento indexado en disco local. Se realiza una recuperación inicial por similitud coseno seleccionando los 10 fragmentos candidatos (Top-K=10).

4. RE-RANKING DE DOS ETAPAS (CROSS-ENCODER)

Los sistemas de recuperación tradicionales basados únicamente en embeddings a veces recuperan fragmentos que tienen similitud de vocabulario pero que no responden en absoluto a la intención de la pregunta clínica.

Para solucionar esto, añadimos una segunda etapa de filtrado mediante un modelo Cross-Encoder ('cross-encoder/ms-marco-MiniLM-L-6-v2'). A diferencia del Bi-Encoder (que procesa la pregunta y el texto por separado), el Cross-Encoder evalúa la consulta del veterinario y cada fragmento simultáneamente en su red de atención. Esto calcula una puntuación de coincidencia exacta y reordena los 10 fragmentos iniciales. Solo los 3 mejores fragmentos con el mayor puntaje de Cross-Encoder se envían a la ventana de contexto del LLM. Esto descarta casi al 100% el contexto ruidoso o irrelevante.

5. INTEGRACIÓN DE HERRAMIENTAS CLÍNICAS (MCP)

Los LLMs nativamente fallan con frecuencia en operaciones matemáticas básicas. En un entorno de salud, un error en una dosificación puede ser crítico.

Para asegurar total fiabilidad, implementamos un servidor bajo el estándar abierto Model Context Protocol (MCP) utilizando FastMCP en Python. Esto expone funciones de cálculo deterministas externas que el LLM puede invocar de forma autónoma:

- `calcular_dosis`: Calcula miligramos y mililitros basados en el principio activo, concentración y peso del paciente.
- `consultar_vacunas`: Devuelve las tablas oficiales de vacunación en base a la edad de la mascota.

Al delegar estas operaciones a funciones Python tradicionales, combinamos la fluidez conversacional del LLM con la precisión e infalibilidad de la programación estructurada.

6. DECISIONES CLAVE DE DISEÑO Y TRADE-OFFS

1. Groq + Llama 3.3 70B: Proporciona razonamiento lógico de alto nivel y excelente soporte en español con latencias de inferencia sumamente bajas (gracias a los procesadores LPU de Groq).
2. ChromaDB Persistente: Nos permite indexar, guardar y leer vectores de embeddings en disco local sin costos adicionales, con soporte nativo de metadatos y filtrado ágil.
3. Validación en LangGraph (Loop de Agentes): Si la información disponible en la base de datos es insuficiente para responder de forma segura, el Writer Agent detiene la generación, activa la bandera 'missing_info' y regresa al Retriever para una segunda búsqueda dirigida, evitando responder con mentiras o adivinaciones.

Firma del Desarrollador Principal

Firma del Evaluador Académico

Ingeniería de Software / IA

Sustentación de Proyecto Final